

AtomicInteger ▼

ATOMICINTEGER

Write a Java program to demonstrate how to use the AtomicInteger class to provide atomic access to a shared integer variable.

Output:
Shared value: 20000

This confirms that the two threads were able to increment the shared value

```
10 Thread t2 = new Thread() -> {
11     for (int i = 0; i < 1000; i++) {
12         resource.increment();
13     }
14 }, "Thread-2");
15 t1.start();
16 t2.start();
17 t1.join();
18 t2.join();
19 System.out.println("Final Count: " + resource.getCount());
20 System.out.println("Expected Count: 2000");
21 }
22 }
23 class AtomicCounter {
24     private AtomicInteger count = new AtomicInteger(0);
25     public void increment() {
26         count.incrementAndGet();
27     }
28
29     public int getCount() {
30         return count.get();
31     }
32 }
```

OUTPUT

```
Final Count: 2000
Expected Count: 2000
```

Your INPUT goes
here!

Sahred ▼

SAHRED

Write a Java program to demonstrate how to use synchronization to prevent multiple threads from accessing a shared resource simultaneously.

```
1 public class Main {
2     public static void main(String[] args) throws InterruptedException {
3         SharedResource resource = new SharedResource();
4         Thread t1 = new Thread() -> {
5             for (int i = 0; i < 1000; i++) {
6                 resource.increment();
7             }
8         }, "Thread-1");
9
10        Thread t2 = new Thread() -> {
11            for (int i = 0; i < 1000; i++) {
12                resource.increment();
13            }
14        }, "Thread-2");
15
16        t1.start();
17        t2.start();
18
19        t1.join();
20        t2.join();
21    }
22 }
```

OUTPUT

```
Final Count: 2000
Expected Count: 2000
```

Your INPUT goes
here!